

Nombre: Jeremy Alexander Ramírez Panameño

Carnet: 202505989

Chapín Warriors

Sistema de control para gestionar misiones de rescate y extracción de recursos en ciudades representadas como mallas de celdas, usando robots (ChapinRescue y ChapinFighter) que se desplazan mediante búsqueda en anchura (BFS) sobre estructuras de datos propias.

Estructura del proyecto

```
IPC2_PROYECT01_2026/
├── Estructuras/
│   ├── Nodo.cs
│   ├── ListasDoblementeEnlazada.cs
│   ├── Cola.cs
│   └── Pila.cs
├── Modelos/
│   ├── Celda.cs
│   ├── MatrizOrtogonal.cs
│   ├── Ciudad.cs
│   └── Robot.cs           (Robot abstracta, ChapinRescue,
ChapinFighter)
├── NodoSistema/
│   ├── NodoSistema.cs / BuscadorRutas.cs
│   └── Mision.cs
├── Servicios/
│   ├── LectorXml.cs
│   ├── SistemaControl.cs
│   └── GeneradorGraphviz.cs
└── Program.cs             # Menú de consola
```

La regla de dependencia es de un solo sentido: Modelos usa Estructuras , nunca al revés. Estructuras no sabe nada sobre celdas, ciudades ni robots — son TDA reutilizables en cualquier proyecto.

Estructuras de datos propias

Todo el proyecto evita las colecciones nativas de C# (`List` , `Queue` , `Stack`). En su lugar:

ListasDoblementeEnlazada

Lista enlazada con punteros `Anterior` / `Siguiente` en cada `Nodo` , y referencias directas a `Cabeza` y `Cola` .

Método	Función
<code>AgregarInicio(dato)</code>	Inserta al inicio (cabeza)
<code>AgregarFinal(dato)</code>	Inserta al final (cola), preservando orden de lectura
<code>EliminarInicio()</code>	Saca y devuelve el primero
<code>EliminarFinal()</code>	Saca y devuelve el último
<code>Eliminar(dato)</code>	Busca por valor y elimina (usado para sobreescritura por nombre)
<code>obtenerporindice(i)</code>	Acceso tipo arreglo (recorriendo desde la cabeza)
<code>estavacia()</code> / <code>obtenertamano()</code>	Consultas básicas

Cola (FIFO)

Compone una `ListasDoblementeEnlazada` internamente. `encolar` inserta al inicio de la lista; `desencolar` saca del final — el extremo opuesto al de inserción, que es lo que garantiza el orden FIFO. Es la estructura que dirige la exploración por niveles del BFS.

Pila (LIFO)

También compone una `ListasDoblementeEnlazada` , pero `apilar` / `desapilar` usan el **mismo** extremo (la cabeza). Se usa en `Mision.ReconstruirRutaEnOrden` para invertir el camino: como `NodoRuta.Padre` solo apunta hacia atrás (del destino a la entrada), apilar en ese orden y luego desapilar devuelve la ruta en el orden correcto (entrada → destino).

Modelo del dominio

Celda

Representa una posición de la malla. Guarda `Fila` , `Columna` , `Tipo` (enum `TipoCelda` : `Intransitable` , `Camino` , `Entrada` , `UnidadCivil` , `Recurso`), `CapacidadUnidadMilitar` (nullable — `null` si no hay unidad militar), y `Visitado` (bandera usada por el BFS).

Métodos de transitabilidad:

- `EsTransitableParaRescate()` — `false` si es intransitable, recurso, o tiene unidad militar.
- `EsTransitableParaFighter(capacidadDisponible)` — `false` solo si es intransitable, o si tiene unidad militar con capacidad mayor o igual a la disponible.

MatrizOrtogonal

Es una "lista de listas": una `ListasDoblementeEnlazada` de filas, donde cada fila es a su vez una `ListasDoblementeEnlazada` de `Celda`. Expone:

- `AgregarFila(contenido, numeroFila)` — interpreta cada carácter (`*` , `,` `E` , `C` , `R`) y construye las celdas.
- `AsignarUnidadMilitar(fila, columna, capacidad)` — se aplica después de cargar todas las filas.
- `ObtenerCelda(fila, columna)` — acceso con validación de límites.
- `ObtenerVecinos(fila, columna)` — hasta 4 celdas ortogonales (arriba/abajo/izquierda/derecha), sin usar arreglos nativos.
- `ReiniciarVisitados()` — limpia la bandera `Visitado` de toda la malla antes de cada búsqueda nueva.

Ciudad

Compone una `MatrizOrtogonal` y mantiene listas de acceso rápido a las celdas especiales, llenadas por `EscanearCeldasEspeciales()` (se llama una sola vez, después de que la malla ya está completamente cargada):

- `ObtenerEntradas()`
- `ObtenerCiviles()`
- `ObtenerRecursos()`

`Filas / Columns` son propiedades de solo lectura que delegan a `Malla.TotalFilas / TotalColumns`, para no duplicar el dato.

Robot (jerarquía con herencia)

`Robot` es **abstracta** y define lo común (`Codigo` , posición) más dos miembros que cada subclase debe implementar:

```
public abstract bool PuedeAtravesar(Celda celda, int capacidadDisponible);  
public abstract int CapacidadInicial();
```

- **ChapinRescue** — PuedeAtravesar delega a EsTransitableParaRescate(). CapacidadInicial() devuelve 0 (no aplica).
- **ChapinFighter** — tiene CapacidadCombate. PuedeAtravesar delega a EsTransitableParaFighter(capacidadDisponible). CapacidadInicial() devuelve CapacidadCombate.

Este diseño permite que BuscadorRutas llame robot.PuedeAtravesar(...) sin necesitar saber qué tipo de robot es — polimorfismo puro.

Mision

Guarda el contexto de una misión (Tipo, CiudadSeleccionada, RobotSeleccionado, CeldaEntrada, CeldaDestino) y el resultado (Exitosa, RutaResultante).

El constructor valida que la entrada sea realmente de tipo Entrada, y que el destino coincida con el tipo esperado según la misión (UnidadCivil para Rescate, Recurso para Extracción); si no, lanza ArgumentException.

Ejecutar() llama a BuscadorRutas.BuscarRuta(...) y, si hay resultado, reconstruye la ruta en el orden correcto usando una Pila (ver arriba).

Algoritmo de búsqueda de rutas

BuscadorRutas.BuscarRuta(malla, entrada, destino, robot) implementa BFS clásico apoyado en Cola:

1. malla.ReiniciarVisitados() — limpia el estado de la búsqueda anterior.
2. Se encola un NodoRuta inicial con la celda de entrada y robot.CapacidadInicial().
3. Mientras la cola no esté vacía: se desencola, se compara contra el destino, y si no coincide se exploran sus vecinos con malla.ObtenerVecinos().
4. Por cada vecino no visitado, se pregunta robot.PuedeAtravesar(vecino, capacidadActual) — aquí es donde el polimorfismo decide la regla correcta según el tipo de robot.
5. **Actualización de poder:** si el vecino tiene unidad militar, se resta su capacidad de la capacidad que trae ese nodo específico del camino — no de la capacidad original del robot. Esto es importante porque el robot puede enfrentar varias unidades a lo largo de la ruta, y cada pelea reduce lo que le queda disponible para la siguiente. Esa capacidad "gastada" viaja en el propio NodoRuta, así que no contamina otras ramas de exploración del BFS.
6. Si la cola se vacía sin llegar al destino, se devuelve null → la Mision queda como "Misión Imposible".

NodoRuta guarda CeldaActual , Padre (para reconstruir el camino) y CapacidadRestante .

Formato del archivo XML

```
<?xml version="1.0"?>
<configuracion>
  <listaCiudades>
    <ciudad>
      <nombre filas="valorEntero"
columnas="valorEntero">valorAlfanumerico</nombre>
      <fila numero="valorEntero">"contenido de m caracteres"</fila>
      <!-- una etiqueta <fila> por cada fila de la ciudad -->
      <unidadMilitar fila="valorEntero"
columna="valorEntero">capacidad</unidadMilitar>
      <!-- cero o mas unidades militares -->
    </ciudad>
    <!-- una o mas ciudades -->
  </listaCiudades>
  <robots>
    <robot>
      <nombre tipo="ChapinRescue">codigo</nombre>
    </robot>
    <robot>
      <nombre tipo="ChapinFighter" capacidad="valorEntero">codigo</nombre>
    </robot>
  </robots>
</configuracion>
```

Caracteres válidos dentro de <fila> : * (intransitable), (camino), E (entrada), C (unidad civil), R (recurso). El contenido viene delimitado por comillas literales ("... "), que LectorXml retira antes de pasarlo a MatrizOrtogonal.AgregarFila .

LectorXml usa XmlDocument (API nativa de System.Xml) con navegación por rutas XPath (SelectSingleNode , SelectNodes) y expone dos métodos: CargarCiudades(ruta) y CargarRobots(ruta) , cada uno devolviendo una ListasDoblementeEnlazada .

Sistema de control

SistemaControl mantiene las listas totales de ciudades y robots a lo largo de toda la ejecución, y expone:

```
bool CargarConfiguracion(string ruta, out string mensajeError)
```

Lógica de sobreescritura: al cargar un archivo, cada ciudad/robot nuevo se compara por nombre/código contra los ya existentes (`AgregarOActualizarCiudad` , `AgregarOActualizarRobot`). Si ya existe uno con el mismo nombre, se elimina el viejo (`Eliminar` , por igualdad de referencia) y se agrega el nuevo en su lugar — así cargar varios archivos de configuración actualiza en vez de duplicar.

Manejo de excepciones: `CargarConfiguracion` envuelve todo el proceso en `try/catch` , capturando `XmlException` (formato XML inválido), `FileNotFoundException` / `DirectoryNotFoundException` (archivo o ruta inexistente), y `Exception` como respaldo general — el programa nunca se cierra abruptamente por un archivo mal formado o inexistente; el error se reporta a través del parámetro `out mensajeError` .

Generación de gráficas (Graphviz)

`GeneradorGraphviz` construye un archivo `.dot` con una tabla HTML (`<TABLE>`) donde cada celda de la malla es un `<TD>` con su propio color de fondo:

Categoría	Color
Inicio (entrada usada)	Verde
Objetivo (civil/recurso destino)	Rojo
Camino recorrido	Amarillo
Combate realizado (unidad militar derrotada en la ruta)	Naranja
Intransitable	Gris oscuro
Entrada no usada / civil o recurso no elegido	Tonos claros distintivos

`GenerarImagen(rutaDot, rutaPng, out mensajeError)` invoca el ejecutable `dot` de Graphviz mediante `System.Diagnostics.Process` , capturando la salida de error estándar. Si `dot` no está instalado o no está en el `PATH` , se captura `Win32Exception` y se informa con un mensaje claro en vez de que el programa truene.

Requisito: Graphviz debe estar instalado en el sistema (`sudo apt install graphviz` en Linux) para que esta parte funcione.

Interfaz de usuario

Menú de consola con 5 opciones, implementado en `Program.cs` :

1. **Cargar archivo de configuración XML** — pide una ruta y llama a `SistemaControl.CargarConfiguracion`.
2. **Consultar / Seleccionar ciudad y robot** — lista las ciudades y robots cargados, guarda la selección en variables `static` que persisten entre iteraciones del menú.
3. **Ejecutar misión** — deduce el tipo de misión según la subclase del robot elegido (`ChapinRescue` → Rescate, `ChapinFighter` → Extracción) usando el operador `is`; valida que la ciudad tenga los objetivos correspondientes; solicita elegir entrada y destino solo si hay más de una opción disponible (si hay una sola, se usa directamente sin preguntar).
4. **Mostrar resultado y generar gráfica** — imprime el resultado de la última misión ejecutada y llama a `GeneradorGraphviz` para producir la imagen.
5. **Salir**.